

CAREER-NAVIGATOR: MACHINE LEARNING DRIVEN EMPLOYMENT FORECASTING

Capstone-I project report submitted

by the student of

Hybrid UG program in Computer Science & Data Analytics

Aumkesh Chaudhary

Roll No. 2312res201

Group No. 11

INDIAN INSTITUTE OF TECHNOLOGY PATNA
BIHTA - 801106, INDIA

Date:

25/06/2024

Declaration

I hereby declare that this submission is my own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Date: 25/06/2024

Aumkesh Chaudhary,
Roll no.: 2312res201,
group: 11

A handwritten signature in black ink, appearing to read 'Aumkesh Chaudhary', written in a cursive style.

Summary of the Project

Anticipated to refine candidates' assessment, CareerNavigator, a forthcoming machine learning model, aims to provide reliable predictions on candidates' employability through comprehensive analysis. The development of the models begins with acquiring the dataset and conducting meticulous data pre-processing to lay a robust foundation. Then comes the model selection, model training, model evaluation, and deployment. Each stage is critical to ensuring the model's accuracy and reliability. We have efficiently distributed our workload, and below are the detailed work breakdowns of each team member.

- **Harshit Kumar:** K-Nearest Neighbour (KNN)
- **Ayush Anand:** Support Vector Machine (SVM)
- **Aditya Anand Singh:** Kernel SVM and Random Forest Classification
- **Md Tanveer:** Naive Bayes and CAP Curve
- **Aumkesh Chaudhary:** Data Preprocessing, Logistic Regression, and Decision Tree Classification

As we approach the end of this project, our collaborative efforts are propelling us towards our objective. CareerNavigator endeavours to transform the job search, making it fairer and more efficient for candidates. We are excited to see our project making a real impact.

Contents

1. Chapter 1: Introduction
2. Chapter 2: Dataset
3. Chapter 3: Importing and Cleaning the Dataset
4. Chapter 4: Categorical Encoding
5. Chapter 5: Splitting and Feature Scaling
6. Chapter 6: Logistic Regression
7. Chapter 7: Decision Tree Classification
8. Chapter 8: Validation and Deployment
9. Chapter 9: Conclusion and Future work

Chapter 1

Introduction

The primary goal of this project is to develop a robust Machine-Learning model capable of predicting the employability status of the candidates. In today's competitive job market, understanding what the recruiters desire has become quite a tough task. By leveraging the Machine-learning algorithms, candidates can potentially gain insights into the specific factors influencing their employability status, enabling them to make informed decisions about their career paths. Furthermore, a more accurate prediction of employability can help candidates focus on their efforts in areas that matter the most to potential employers.

Driven by the aforementioned motive, I have embarked on an endeavour to bring this project to fruition. I have procured a dataset from 'Kaggle', leveraging it extensively in the development of the Machine-learning model. The development process starts with importing and cleaning the dataset, then comes the Categorical Encoding part, and the whole Data pre-processing part ends with the Feature scaling leading toward the Training, Testing, and Validation of the model. We will delve deep into these topics in greater detail in the chapters ahead, offering a comprehensive exploration of each.

I will be leveraging the indispensable 'Scikit-learn' (sklearn) library, crucial for our machine learning model. The scikit-learn library empowers us to perform encoding with 'LabelEncoder' and 'OneHotEncoding', effortlessly split data into training and testing sets using 'Train_Test_Split', and effectively scale features with 'StandardScaler'. The scikit-learn library will help us throughout this project's development, be it in regression, classification, or validation.

The development of this Machine-Learning model will be segmented into distinct parts, each addressing specific components and stages of the process. For instance, we will be working on data pre-processing, choosing the best model, and testing the accuracy of the model. By organizing the development into these distinct parts, we ensure a systematic and thorough approach to building a robust and accurate machine-learning model.

Chapter 2

DATASET

The dataset has been procured from ‘Kaggle’. It contains 73,462 rows and 15 columns. The columns are described below:

- **Age**: age of the applicant, >35 years old or <35 years old (*categorical*)
- **Accessibility**: whether the applicants have access to special tools or features that make it easier for people with disabilities to use computers or software (*categorical*)
- **EdLevel**: education level of the applicant (Undergraduate, Master, PhD...) (*categorical*)
- **Gender**: gender of the applicant, (Man, Woman, or NonBinary) (*categorical*)
- **Employment**: whether the applicant has ever been employed or not (*categorical*)
- **MainBranch**: whether the applicant is a professional developer (*categorical*)
- **MentalHealth**: whether the applicant is mentally fit or not (*categorical*)
- **YearsCode**: how long the applicant has been coding (*integer*)
- **YearsCodePro**: how long the applicant has been coding in a professional context, (*integer*)
- **Country**: Nationality of the applicant (*categorical*)
- **PreviousSalary**: the applicant's previous job salary (*float*)
- **HaveWorkedWith**: specific technical skills possessed by the applicant (*categorical*)
- **ComputerSkills**: number of computer skills known by the applicant (*integer*)
- **Employed**: target variable, whether the applicant has been hired (*categorical*)

Source: <https://www.kaggle.com/datasets/ayushtankha/70k-job-applicants-data-human-resource>

The dataset encompasses a comprehensive survey of individuals within the computer science domain, capturing various demographics, professional, and personal attributes. It includes information such as age, education level, employment status, mental health status, years of coding experience, country of residence, previous salary, and computer skills. It provides valuable insights into the employment trend which is perfectly suited for our machine learning model.

Chapter 3

Importing and Cleaning the Dataset

In this chapter, we delve into the process of importing and cleaning the dataset obtained for our study. For our Machine-Learning model, we have chosen 'python' as our primary programming language. Therefore, Firstly, I have imported some necessary libraries - 'pandas', 'numpy', and 'matplotlib' - which we are going to use throughout our project.

- **Reading the dataset:**

I have created a DataFrame named 'Dataset' using the pandas library and read the dataset. After reading the dataset, I used the method '.info()' to get an overview of the dataset such as the datatype of the columns and null value.

- **Changing the Datatype of the column:**

There is a column named 'PreviousSalary' which contains the values stored as float. So, we will have to convert it into integer value as it would create difficulty in the feature scaling which we will be talking about in the upcoming chapter. I have used '.replace()' function to convert the datatype of the column.

- **Categorizing the Countries into Continents:**

There are more than 190 countries in the Dataset. So, it would become challenging to encode all of them as the categorical encoding creates vectors for each category which drastically increases the dimensionality of the dataset making it extremely difficult to handle and sometimes resulting in crashing the program. Therefore, I have categorized each country by its respective continents, streamlining the dataset for easier management. I had to define a function 'Countries', and I used 'if' and 'else' statements to categorize the countries into each continent, and then created a column named continent in the dataframe (i.e., 'Dataset') using that function.

- **Dropping the Columns:**

Some columns in this dataset do not have much significance for our model such as 'HaveWorkedWith' which again, if kept, can increase the dimension excessively and the other one is column 'YearsCodePro', which is not essential as we already have the column 'YearsCode'. I have also dropped the column 'Country' as we have created a column for 'Continents'.

Chapter 4

Categorical Encoding

It is Probably the most important part of the whole data pre-processing. As we know the computer does not understand the string values and it cannot perform any comparative operations or studies on it, we are required to convert all the string values of the dataset into binary.

In this dataset of ours, we have two kinds of Categorical columns: columns with two categories and columns with more than two categories. For the columns with two categories such as Age, Accessibility, MentalHealth, and MainBranch, I have used LabelEncoder which converts two categories into binary values as ‘1’ and ‘0’. While, for the columns having categories more than one such as EdLevel, Gender, and Continent, I have used OneHotEncoding as it vectorizes the categories creating vector columns for each category. For the rest of the columns containing integer values, I have left them as they are.

	0	1	2	3	4	5	6	7	8	9	...	14	15	16	17	18	19	20	21	22	23
0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	1.0	0.0	0.0	8.0	9792.0	9.0	0.0
1	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	1.0	1.0	0.0	5.0	4200.0	9.0	0.0
2	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	1.0	0.0	0.0	6.0	4200.0	9.0	1.0
3	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	1.0	1.0	0.0	0.0	3.0	3108.0	9.0	0.0
4	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	1.0	0.0	0.0	5.0	3108.0	9.0	0.0

5 rows x 24 columns

Source: <https://colab.research.google.com/drive/1bbNuUBh95bQFtnNBEI92gjpfnruRhvq6>

In the above image, extracted from the colab file forming the foundation of my work, a noticeable transformation is evident: the number of columns has expanded from 14 to approximately 24 following the implementation of OneHotEncoding. Additionally, through LabelEncoding, each categorical column has been converted into binary representations, significantly altering the dataset’s structure, and paving the way for enhanced analysis.

Chapter 5

Splitting and Feature Scaling

Navigating through the data-preprocessing phase, we encounter two pivotal stages: Dataset Splitting and Feature Scaling. Now, we will dive deep into the essence of these two areas which are going to help us immensely in our upcoming work

- **Splitting the Dataset:** we have a fully encoded dataset, and we want to develop a machine-learning model. What we now need is two kinds of datasets, one to train the model and another to test its performance. To get these two datasets, we do not require them from external sources but, we will split our already encoded dataset into a training set and a testing set. To split the dataset, I have used the function `'train_test_split'` from the Scikit-learn library. This function divides the dataset into `'train_set'` and `'test_set'`. I have partitioned the dataset into an 80:20 ratio. In this partition, 80% of data points are allocated to the training set while the remaining 20% are designated to the testing set. But, while doing the splitting part, we could end up with the problem of getting an uneven distribution of the dataset. Therefore, I have used the parameter `'random_state'` to prevent the dataset from uneven or biased distribution.
- **Feature Scaling:** In feature scaling, what we do is normalize the dataset into a specific range. Generally, two types of normalizations are used in feature scaling: `'Min- Max'` or `'Z-Score'`. However, I have used the Z-Score normalization which is the most effective normalization method because of its ability to preserve the original data distribution, effectively handle outliers, and ensure interpretability. In Z-Score normalization, the data points are transformed to fall within the range of -3 to 3, ensuring standardized distribution and facilitating comparisons across variables. To perform the feature scaling part, I have used the `'StandardScaler'` function from the scikit-learn library. Using this function, all data points, excluding the vector columns, undergo scaling to be constrained within the range of -3 to 3.

Chapter 6

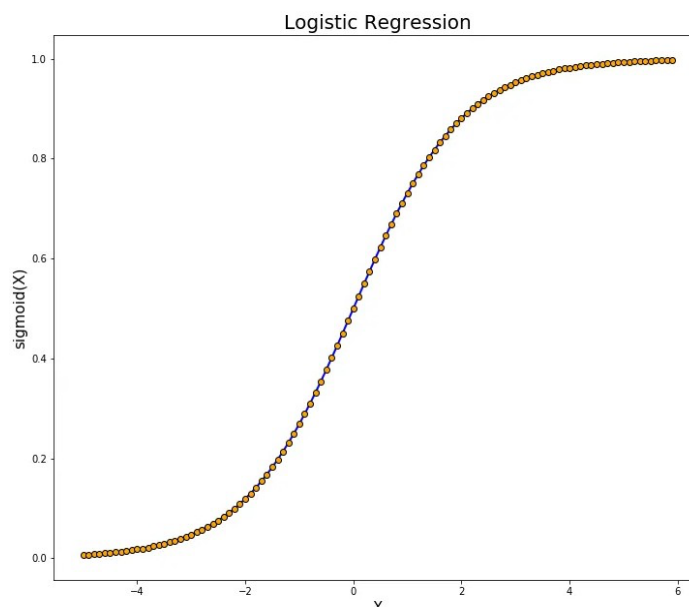
Logistic Regression

Before delving into the logistic regression model, it is crucial to highlight that we have labeled data. This means our task is to predict an output that will be binary, i.e., 'Yes' or 'No'. Given this scenario, we will implement a supervised learning method to address the problem.

For our initial approach, we will develop a logistic regression model. Logistic regression is a robust statistical technique specifically designed for binary classification tasks. Its objective is to predict one of two possible outcomes based on one or more predictor variables. The Logistic regression estimates the probability that a given input point belongs to a specific class. This probabilistic approach makes it ideally suited for our binary classification needs, allowing us to make informed and accurate predictions.

$$\ln \frac{p}{1-p} = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4$$

Here, ' p ' denotes the probability of the dependent variables being in a particular class. ' b ' represents the coefficient of the model. And ' X ' denotes the predictor variables (independent variables) used to predict the probability of ' p '.



Source: <https://towardsdatascience.com/logistic-regression-explained-9ee73cede081>

Continued....

- The provided graph illustrates the logistic regression model's characteristic S-shaped curve, known as the sigmoid function.

To train the Logistic Regression model, I have used the Class 'LogisticRegression' from the scikit-learn library. Below is the code which has been used to train the model.

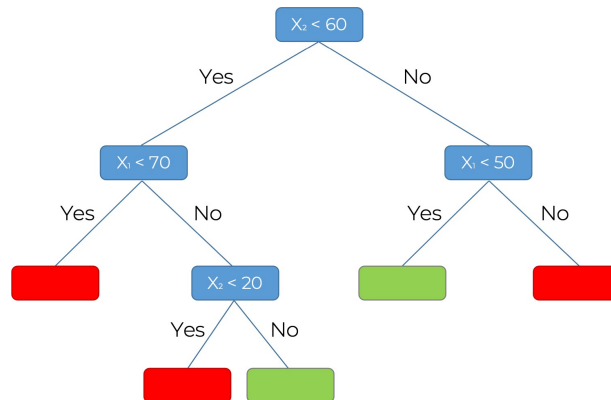
```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(random_state = 0)
classifier.fit(x_train, y_train)
```

▼ LogisticRegression
LogisticRegression(random_state=0)

Source: <https://colab.research.google.com/drive/1bbNuUBh95bQFtnNBEI92gjpfnrhRvq.6#scrollTo=XKgGivglvq93>

Chapter 7

Decision Tree Classification



Source: <https://www.udemy.com/share/101WfW3@iPsaByEcWsmnT7Yz58VIXS9jclr7WxV-NxXamQ9d1Fom1ZzmrWPlesFVKDzTbgQSkA==/>

Decision tree classification is a highly effective and widely adopted machine learning technique for categorizing data into predefined classes. This approach involves constructing a tree-like model in which each node signifies a decision based on a specific feature, each branch illustrates the possible outcomes of these decisions, and each leaf node represents the final class label. The hierarchical structure of decision trees enables systematic and intuitive data partitioning, making it straightforward to interpret the decision-making process at every stage. This clarity not only enhances the accuracy of the classifications but also provides valuable insights into the underlying patterns and relationships within the data.

I have used the 'DecisionTreeClassifier' class from the 'scikit-learn' library to train the Decision Tree Classification model. The code is given below:

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(n_estimators = 100, criterion = 'entropy')
classifier.fit(x_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(criterion='entropy', random_state=0)
```

Source: <https://colab.research.google.com/drive/1bbNuUBh95bQFtnNBEI92gjpfnruRhvq.6#scrollTo=XKgGivglvq93>

Chapter 8

Validation and Deployment

In this section, we will delve into the evaluation of the testing set results and the deployment strategy for our machine learning model.

Model Performance Metrics:

- **Accuracy**: The ratio of correctly predicted instances to total instances. Useful but can be misleading for imbalanced datasets.
- **Confusion Matrix**: A table showing true/false positives and negatives, providing detailed performance insights.
- **Precision, Recall, and F1 Score**: Precision measures the accuracy of positive predictions, recall gauges the model's ability to find all positive instances, and the F1 score harmonizes these metrics into a single value.

Table of Evaluation

Metrics	Logistic Regression	Decision Tree Classification
<u>Accuracy</u> :	78.70414483087185 %	70.70713945416185 %
<u>Confusion Matrix</u> :	[[5262 1506] [1623 6302]]	[4691 2077] [2227 5698]]
<u>F1 Score</u> :	[0.7708196 0.80111867]	[0.68551805 0.72585987]

Continued....

Now, coming to the deployment part, I'm in the middle of deploying the model using Flask on cloud infrastructure. Flask's lightweight but powerful framework is really helpful for managing how the model handles requests and responses—it's efficient and reliable. With Flask, I'm building a RESTful API to make it easy for users to interact with the machine learning model. I am going through each step carefully during deployment to ensure the model performs well and responds quickly when users make predictions.

The code is given below which has been used to create the API:

```
import joblib
import pickle as pkl
from flask import Flask, request, jsonify

app = Flask(__name__)

joblib.dump(classifier, 'classifier.joblib')

model = joblib.load('classifier.joblib')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json(force=True)
    prediction = model.predict(data['input_data'])
    return jsonify({'prediction': prediction.tolist()})

if __name__ == '__main__':
    app.run(debug=True)

* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
```

Source: <https://colab.research.google.com/drive/1bbNuUBh95bQFtnNBEI92gjpfnrhRhvq.6#scrollTo=XKgGivglvq93>

Chapter 9

Conclusion and Future Work

As we draw towards the conclusion of this report, I aim to provide a comprehensive overview encompassing the main conclusions derived from the project's findings. Additionally, an in-depth exploration of the project's future prospects will be provided, outlining potential opportunities for growth, development, and strategic direction moving forward.

I am on an ongoing journey to develop our model the 'CareerNavigator'. So far, I have obtained the dataset and performed data preprocessing. During preprocessing, I have altered and modified the data in many ways. After cleaning, I encoded it using 'LabelEncoder' and 'OneHotEncoding' and split it into training and testing sets. I have Feature-scaled the modified dataset. As our dataset comes labelled, it naturally aligns with the use of supervised learning, making it the ideal fit for our ML model.

For supervised learning, I have used the 'Classification method', since we need to find the discrete value, consisting of seven different components. Out of those seven models, I have made 'two' models: 'Logistic Regression' and 'Decision Tree Classification'. I have evaluated and validated all the models and as a result, found that Aditya's model 'Kernel-Support Vector Machine (Kernel-SVM)' has the highest accuracy and F1 Score which are '79.03083100796297 %' and '[0.76107018 0.81317082]' respectively. Currently, I am on the way to deploying the SVM-trained model to cloud API using Flask.

Regarding future prospects, my plan entails developing a website to integrate this model. This website will feature a user-friendly interface housing a range of academic and personal inquiries, facilitating the model's prediction of employment status, thereby offering valuable insights for career planning and decision-making. I also Plan to extend the website to recruiters to assess the potential candidates more effectively.

References

Referenced study materials for this project:

- **Dataset:** Taken from Kaggle titled '70k+ job Applicants Data',
source: <https://www.kaggle.com/datasets/ayushtankha/70k-job-applicants-data-human-resource/code>
- **The idea of categorizing Countries into continents:**
'Bias_Detection_In_Recruitment_AI_with_LIME' by Donghyeok-Lee', **source:** <https://www.kaggle.com/code/leedonghyeok/1-bias-detection-in-recruitment-ai-with-lime>
- **For the Study of Machine Learning:** 'Udemy',
source: <https://www.udemy.com/share/101WfW3@iPsaByEcWsmnT7Yz58VIXS9jclr7WxV-NxXamQ9d1Fom1ZzmrWPlesFVKDzTbgQSkA==/>